CA-BEFORE-MID — Weeks 1–8 · 17 Aug – 11 Oct 2026 · MIDTERM (W8)

Scope: Weeks 1–8 (17 Aug – 11 Oct 2026) — MIPS ISA, single-cycle, multi-cycle, FSM/microprogrammed control, pipeline intro and hazards I/II, ending at the Week 8 midterm. **Siblings:** `CA-NAV.md` (master map) · `CA-AFTER-MID.md` (Weeks 9–15). Week-colored headers below match the NAV schedule. **Exam split:** 60% diagram accuracy + numerical precision, 40% theory. Drawing from memory is the skill. **Exam order:** diagram/numerical questions first, theory second.

At a glance (W1–W8)

W	Dates	Variant	File	Topics	Chapter
W1	17–23 Aug	P2	BEFORE	MIPS ISA & single-cycle intro	01 + 02
W2	24–30 Aug	P2	BEFORE	Multi-cycle datapath	03
W3	31 Aug–06 Sep	P2	BEFORE	Multi-cycle FSM control	03
W4	07–13 Sep	P0	BEFORE	Microprogramming & pipeline intro	04
W5	14–20 Sep	P0	BEFORE	Pipeline hazards I (structural/data)	04
W6	21–27 Sep	P0	BEFORE	Pipeline hazards II (detection/branch pred)	04
W7	28 Sep–04 Oct	P1	BEFORE	Midterm revision	01–04
W8	05–11 Oct	MIDTERM	BEFORE	Exam week — no new material	—

Tier key: **P0** = no exam pressure (front-load new material) · **P1** = light revision · **P2** = drill-heavy week.

W1 — MIPS ISA & Single-Cycle Datapath · 17–23 Aug · P2 · 6 hrs

Banner: Rotation CA (Mon+Tue) · Tier P2 · Time budget 6 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 1	lines 17–30
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W1	lines 13–19
Week manifest	`weeks/CA-W1.md`	full file
Chapter breakdowns	`03-Chapter-Breakdowns/01-…` + `02-…`	embedded below

Definitions (verbatim)

Term	Definition
ISA	The interface between hardware and software, defining the instructions, registers, memory addressing, and data types that a processor supports.
Single-cycle Datapath	A processor implementation where every instruction executes in exactly one clock cycle, with the cycle time determined by the longest instruction (typically lw).
R-type instruction	An instruction format where the operation is performed on register operands and the result is stored in a register, using fields for opcode (6 bits), rs (5), rt (5), rd (5), shamt (5), and funct (6).
I-type instruction	An instruction format that includes an immediate value, using fields for opcode (6 bits), rs (5), rt (5), and immediate (16 bits).
J-type instruction	An instruction format used for unconditional jumps, using fields for opcode (6 bits) and target address (26 bits).

Register convention	The agreed-upon usage of registers in the MIPS ISA, including $zero (constant 0), $s0-$s7 (saved), $t0-$t9 (temporary), $a0-$a3 (arguments), $v0-$v1 (return values), $sp (stack pointer), $fp (frame pointer), $ra (return address).
Control signals	RegDst, RegWrite, ALUSrc, ALUOp, MemRead, MemWrite, MemtoReg, Branch, Jump.

Formulas (verbatim)

Formula	Statement
CPU Time	CPU Time = Instruction Count × CPI × Cycle Time
Speedup	Speedup = Old Time / New Time
CPI	The average number of clock cycles per instruction.

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#1** single-cycle datapath (canonical) · **#9** MIPS instruction formats
Numericals	`Numerical-Book.md` **#1** CPI from mix · **#2** CPU time
Tricky	`Top-10-Tricky-Concepts.md` **#6** multi-cycle vs single-cycle paradox
Top-100	`Top-100-Questions.md` **#1–#10** (ISA & formats)

Books · Chapters · Media

Resource	Where
Hennessy & Patterson CAQA	App A (ISA principles)
Patterson & Hennessy COD	Ch 2 (RISC-V parallels; syllabus is MIPS)
Sarangi (IITD)	Ch 3 (SimpleRisc/RISC-V — parallel reading only)
YouTube	Sarangi "Basic Computer Architecture" playlist (48 vids, ch 8–12) · CS61C (UC Berkeley)
MOOC	NPTEL noc25_cs83 (Sarangi)

Fear-Killer Pack CA-W1 (verbatim)

Resources: H&P CAQA App A (ISA principles) • P&H COD Ch 2 (RISC-V ISA parallels; syllabus is MIPS)

1. Decode `0xAC010004` into assembly and encode `addi $s0, $s1, -8` into machine code. Explain when sign extension happens and why I-type immediates are sign-extended.
2. Write the complete MIPS register convention table from memory: numbers and roles for $zero, $v0-$v1, $a0-$a3, $t0-$t9, $s0-$s7, $sp, $fp, $ra. Which register is hardwired to 0 and why does this enable `move` and `not`?
3. Draw the complete single-cycle datapath from blank paper with every control signal labeled. Trace `lw $t0, 4($s1)` showing every mux selection. Repeat until <15 min.
4. Given component delays (register 20ps, ALU 100ps, memory 200ps), compute the cycle time. Why is `lw` the critical path? Which instructions would be slowed down by a single-cycle design?

Narrative — Week 1 (verbatim)

Topics: MIPS instruction formats (R-type, I-type, J-type); Register conventions; Single-cycle datapath design

Resources: Hennessy & Patterson App. A; Smruti Sarangi (IITD) Ch. 3 (SimpleRisc/RISC-V — parallel reading only; book is not MIPS-based)

Practice: - Decode 10 MIPS instructions into binary (R-type: opcode, rs, rt, rd, shamt, funct) - Draw the single-cycle datapath from memory. Label every wire. Repeat until it takes <15 min

Deliverable: Hand-drawn single-cycle datapath with all control signals labeled

Time budget: 6 hrs

Chapter 1 — Instruction Set Architecture (verbatim)

Weight: ★★★★★

Difficulty: ★★★☆☆

Importance: ★★★★★

Learning Objectives

Explain

- RISC vs CISC philosophy
- MIPS register conventions
- Addressing modes

Compare

- R-type vs I-type vs J-type formats
- Different addressing modes

Draw

- MIPS instruction format diagrams
- Register field positions

Calculate

- Instruction encoding from assembly
- Instruction decoding from machine code

Differentiate

- R-type: opcode + rs + rt + rd + shamt + funct (6 fields)
- I-type: opcode + rs + rt + immediate (4 fields)
- J-type: opcode + target address (2 fields)

Must Memorize

Definitions (Word-for-word)

Instruction Set Architecture (ISA): The interface between hardware and software, defining the instructions, registers, memory addressing, and data types that a processor supports.

R-type instruction: An instruction format where the operation is performed on register operands and the result is stored in a register, using fields for opcode (6 bits), rs (5), rt (5), rd (5), shamt (5), and funct (6).

I-type instruction: An instruction format that includes an immediate value, using fields for opcode (6 bits), rs (5), rt (5), and immediate (16 bits).

J-type instruction: An instruction format used for unconditional jumps, using fields for opcode (6 bits) and target address (26 bits).

Register convention: The agreed-upon usage of registers in the MIPS ISA, including $zero (constant 0), $s0-$s7 (saved), $t0-$t9 (temporary), $a0-$a3 (arguments), $v0-$v1 (return values), $sp (stack pointer), $fp (frame pointer), $ra (return address).

Must Understand

- Why MIPS uses fixed instruction length (32 bits) — simplifies fetch/decode
- Why $zero is hardwired to 0 — enables common operations (move, not, etc.)
- Addressing modes: register, immediate, base/displacement, PC-relative, pseudo-direct

Must Practice

Encoding/Decoding

```
Assembly → Machine Code: add $t0, $s1, $s2
  opcode = 0 (R-type), rs = $s1 (17), rt = $s2 (18), rd = $t0 (8), shamt = 0, funct = 32 (add)
  Binary: 000000 10001 10010 01000 00000 100000
  Hex: 0x02324020

Machine Code → Assembly: 0x8D080000
  opcode = 0x23 (35 = lw), rs = 0x08 ($t0), rt = 0x08 ($t0), immediate = 0x0000
  Assembly: lw $t0, 0($t0)
```

Common Mistakes

1. **Misidentifying instruction format** — check opcode first. If opcode = 0, it's R-type. Otherwise I-type or J-type.
2. **Confusing funct field with opcode** — for R-type, opcode is always 0; the actual operation is in funct.
3. **Register numbering** — $t0 = 8, not 0. Common off-by-8 error.
4. **Immediate sign extension** — I-type immediates are sign-extended to 32 bits for ALU operations.

Past Paper Trends (2019-2024)

- **2019** — Decode 3 instructions, identify format
- **2020** — Compare RISC vs CISC, encode 4 instructions
- **2021** — Register convention question, decode R-type
- **2022** — Addressing modes, encode branch instruction
- **2023** — Full instruction decode table, pseudo-instructions
- **2024** — Compare J-type target calculation vs PC-relative

Frequency: Appears every year. Usually 10-15 marks combined with datapath.

Typical Questions

1. Decode the MIPS instruction `0xAC010004` into assembly
2. Encode `addi $s0, $s1, -8` into machine code
3. Explain the purpose of $zero register
4. Differentiate R-type, I-type, J-type with diagrams
5. Calculate target address for a j instruction given PC

Examiner Expectations

Level	Performance
Pass	Can decode/encode basic instructions
Good	Handles all three formats, understands sign extension
Excellent	Identifies pseudo-instructions, knows all register conventions
Full marks	Encodes/decodes under time pressure with zero errors

Formula Sheet

MIPS Instruction Formats

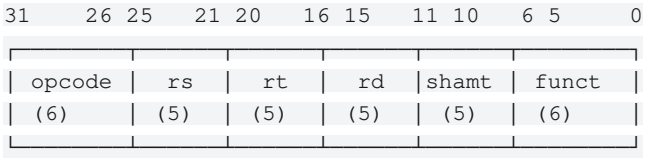
```
R-type: [opcode(6)][rs(5)][rt(5)][rd(5)][shamt(5)][funct(6)] = 32 bits
I-type: [opcode(6)][rs(5)][rt(5)][immediate(16)] = 32 bits
J-type: [opcode(6)][target(26)] = 32 bits
```

Register Numbers

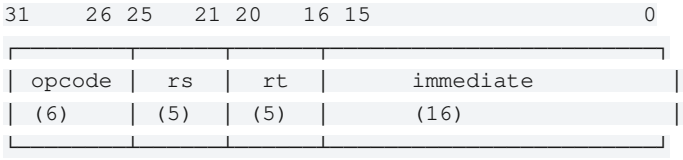
```
$zero = 0   $a0-$a3 = 4-7   $v0-$v1 = 2-3
$t0-$t7 = 8-15   $s0-$s7 = 16-23   $t8-$t9 = 24-25
$sp = 29   $fp = 30   $ra = 31
```

Diagram Sheet

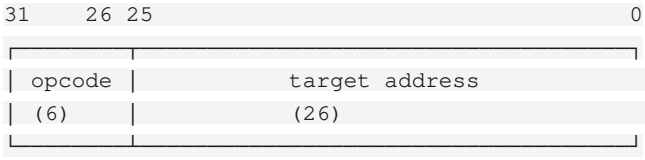
R-type Format



I-type Format



J-type Format



Flashcards

- Q: Which MIPS register is hardwired to 0? → A: $zero (register 0)
- Q: How many bits in a MIPS instruction? → A: 32 bits (fixed length)
- Q: What determines if an instruction is R-type? → A: opcode = 0 (then funct field specifies operation)
- Q: What is the J-type instruction used for? → A: Unconditional jumps
- Q: How is the target address calculated in a j instruction? → A: {PC+4[31:28], target[26:0], 00}

Retrieval Questions (50+)

1. List all MIPS instruction formats with field sizes
2. Encode: `add $s0, $s1, $s2`
3. Decode: `0x8D08FFFC`
4. List register numbers for $t0-$t9
5. List register numbers for $s0-$s7
6. What is the purpose of $sp?
7. What is sign extension? When does it happen?
8. Encode a conditional branch instruction
9. Calculate J-type target from a given address
10. Explain why MIPS uses fixed instruction length 11-50. (Additional practice problems in Numerical Book)

GPA Priority: ▓ Must Win

Chapter 2 — Single-Cycle Datapath (verbatim)

Weight: ★★★★★

Difficulty: ★★★☆☆

Importance: ★★★★★

Learning Objectives

Explain

- How each instruction flows through the datapath

- Role of each control signal

Draw

- Complete single-cycle datapath from memory
- Control unit truth table

Calculate

- Cycle time for single-cycle implementation
- CPI = 1 (always)

Design

- Control logic for each instruction
- Datapath modifications for new instructions

Must Memorize

Single-cycle datapath: A processor implementation where every instruction executes in exactly one clock cycle, with the cycle time determined by the longest instruction (typically lw).

Control signals: RegDst, RegWrite, ALUSrc, ALUOp, MemRead, MemWrite, MemtoReg, Branch, Jump.

Must Draw

- Complete single-cycle datapath: PC → Instruction Memory → Register File → ALU → Data Memory → MUXes back
- Every wire labeled
- Every control signal labeled
- Complete in <15 min

Common Mistakes

1. **Missing the PC+4 adder** — it's separate from the ALU
2. **MUX at register file write address** — uses RegDst to select rt(20:16) vs rd(15:11)
3. **Sign extension for I-type** — immediate goes through sign-extend before ALU
4. **Branch equality check** — uses a separate AND gate (Branch & Zero)

Typical Questions

1. Draw complete single-cycle datapath with control signals
2. Trace `lw $t0, 4($s1)` through datapath — show every mux selection
3. What is the critical path? Why is it lw?
4. Calculate cycle time given component delays
5. Add a new instruction — show datapath modifications

GPA Priority: ▩ Must Win

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 decode/encode + Q3 datapath draw) · def/formula skim · same-problem drill #1, #2
- **Same-problem drill target:** single-cycle datapath draw < 15 min; decode/encode instruction < 30 s
- **Deliverable:** hand-drawn single-cycle datapath with all control signals labeled
- **Trap:** don't over-invest — ISA/single-cycle is background-review (P2), foundation only

---

W2 — Multi-Cycle Datapath · 24–30 Aug · P2 · 6 hrs

Banner: Rotation CA (Mon+Tue) · Tier P2 · Time budget 6 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 2	lines 33–46
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W2	lines 21–26
Week manifest	`weeks/CA-W2.md`	full file
Chapter breakdown	`03-Chapter-Breakdowns/03-…`	embedded below

Definitions (verbatim)

Term	Definition
Multi-cycle Datapath	A processor implementation that breaks instruction execution into multiple steps.
Microprogrammed Control	A control unit implementation that uses microinstructions stored in a control store to generate control signals.
Temporary registers	IR (instruction register — exception, needs a write control signal), MDR (memory data register), A, B (ALU input buffers), ALUOut (holds ALU result).

Formulas (verbatim)

Formula	Statement
Speedup	Speedup = Old Time / New Time
CPI	The average number of clock cycles per instruction.

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#2** multi-cycle datapath w/ IR, MDR, A, B, ALUOut (canonical)
Numericals	`Numerical-Book.md` **#1** CPI from mix · **#2** CPU time
Tricky	`Top-10-Tricky-Concepts.md` **#6** multi-cycle vs single-cycle paradox
Top-100	`Top-100-Questions.md` **#11–#25** (datapath)

Books · Chapters · Media

Resource	Where
Hennessy & Patterson CAQA	App C
Patterson & Hennessy COD	Ch 4
Sarangi (IITD)	Ch 9 · CA textbook Ch 5

Fear-Killer Pack CA-W2 (verbatim)

Resources: H&P CAQA App C • P&H COD Ch 4

1. Trace `lw $t0, 4($s1)` through the multi-cycle datapath step-by-step. List the register transfers and control signals for each state. Then repeat for `beq $t0, $t1, label` — which states differ and why?
2. A single-cycle processor has cycle time 10ns. The same design in multi-cycle has states: IF=2ns, ID=1ns, EX=2ns, MEM=3ns, WB=1ns. For an instruction mix of 25% lw, 20% sw, 40% R-type, 15% branch, calculate the speedup of multi-cycle over single-cycle.
3. Draw the multi-cycle datapath from memory, labeling IR, MDR, A, B, and ALUOut. Why are these temporary registers needed? Which one is the exception and requires a write control signal?

Narrative — Week 2 (verbatim)

Topics: Multi-cycle vs single-cycle; Multi-cycle datapath for R-type, lw, sw, branch

Resources: Sarangi Ch. 9; CA textbook Ch.5

Practice: - Trace R-type, lw, sw, and branch through the multi-cycle datapath step by step - Draw the multi-cycle datapath from

memory. Repeat until flawless in 15 min

Deliverable: Multi-cycle datapath diagram with state labels for each instruction type

Time budget: 6 hrs

Chapter 3 — Multi-Cycle Datapath (verbatim)

Weight: ★★★★☆

Difficulty: ★★★★☆

Importance: ★★★★★

Learning Objectives

Explain

- Why multi-cycle is faster than single-cycle
- How state transitions control instruction flow

Compare

- Single-cycle vs multi-cycle performance

Draw

- Complete multi-cycle datapath from memory
- State diagrams for each instruction

Trace

- R-type, lw, sw, branch through multi-cycle steps

Must Memorize

Multi-cycle datapath: A processor implementation that breaks instruction execution into multiple steps, each taking one clock cycle, allowing different instructions to take different numbers of cycles.

Must Draw

- Multi-cycle datapath: shared ALU, instruction register, memory data register, A/B registers, ALUOut register
- Complete in <15 min

Common Mistakes

1. **Missing the instruction register (IR)** — holds instruction during execution
2. **Missing the memory data register (MDR)** — buffers data from memory
3. **State 0 return** — every instruction must return to state 0 (instruction fetch)
4. **Register file write timing** — write happens at end of cycle to avoid read-after-write hazard

Typical Questions

1. Draw complete multi-cycle datapath
2. Trace `add $t0, $s1, $s2` through all states
3. Compare cycle time: single-cycle vs multi-cycle
4. Why is multi-cycle faster? (shorter cycle time)
5. What are states for each instruction type?

GPA Priority: ▓ **Must Win**

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 trace + Q3 draw) · def/formula skim · same-problem drill #1, #2
- **Same-problem drill target:** multi-cycle datapath draw < 15 min; trace `lw` step-by-step with states
- **Deliverable:** multi-cycle datapath diagram with state labels for each instruction type
- **Trap:** don't drop the IR/MDR registers; register-file write at end of cycle

---

W3 — FSM Control Unit · 31 Aug–06 Sep · P2 · 6 hrs

Banner: Rotation CA (Mon+Tue) · Tier P2 · Time budget 6 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 3	lines 49–64
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W3	lines 28–33
Week manifest	`weeks/CA-W3.md`	full file
Chapter breakdown	`03-Chapter-Breakdowns/03-…` (M2 gap: FSM/microprogrammed control is pack-level only)	reference (embedded W2)

Definitions (verbatim)

Term	Definition
Microprogrammed Control	A control unit implementation that uses microinstructions stored in a control store to generate control signals.
FSM (hardwired) control	Control unit implemented as a state machine — faster, less flexible than microprogramming.

Formulas (verbatim)

Formula	Statement
Speedup	Speedup = Old Time / New Time

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#6** FSM state diagram for multi-cycle control (canonical)
Numericals	`Numerical-Book.md` **#1** CPI from mix
Tricky	`Top-10-Tricky-Concepts.md` **#6** multi-cycle vs single-cycle paradox
Top-100	`Top-100-Questions.md` **#15–#25** (FSM/control/CPI)

Books · Chapters · Media

Resource	Where
Patterson & Hennessy COD	App C (mapping control to hardware)
Past papers	FAST / NED

Fear-Killer Pack CA-W3 (verbatim)

Resources: P&H COD App C (mapping control to hardware) • past papers from FAST/NED

1. Design the FSM state diagram for the multi-cycle control unit covering R-type, lw, sw, and branch instructions. Show all state transitions with conditions. Provide the state encoding table.
2. Every instruction must return to state 0 (instruction fetch). What breaks if a state transition is missing? Trace `sw $t1, 8($t2)` and show exactly where an incomplete diagram would stall the processor.
3. Contrast FSM-based (hardwired) control vs microprogrammed control: speed, flexibility, control store size. When does the FSM approach win and when does microprogramming win?

Narrative — Week 3 (verbatim)

Topics: FSM for multi-cycle control; State transitions for each instruction; ASM charts

Resources: Sarangi Ch. 9; past papers from FAST/NED

Practice: - Draw FSM state diagram for R-type, lw, sw, branch - Ensure every state transition is complete (missing wait states = lost marks) - Contrast FSM control vs microprogrammed control

Trap alert: Most students draw incomplete state transitions. Every instruction must return to the same initial state (state 0/ instruction fetch). Verify this.

Deliverable: FSM state diagram for R-type, lw, sw, branch — every transition labeled

Time budget: 6 hrs

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 state diagram + Q2 transition completeness) · def/formula skim · same-problem drill #1
- **Same-problem drill target:** FSM state diagram complete with all transitions to state 0; spot an incomplete diagram in < 30 s
- **Deliverable:** FSM state diagram for R-type, lw, sw, branch — every transition labeled
- **Trap:** missing transitions to state 0 = lost marks (Trap alert above)

W4 — Microprogrammed Control & Pipeline Intro · 07–13 Sep · P0 · 6 hrs

Banner: Rotation CA (Mon+Tue) · Tier P0 · Time budget 6 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 4	lines 68–81
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W4	lines 35–40
Week manifest	`weeks/CA-W4.md`	full file
Chapter breakdown	`03-Chapter-Breakdowns/04-…` (hazards only; ILP = pack-level gap M3)	reference (embedded W5)

Definitions (verbatim)

Term	Definition
Microprogrammed Control	A control unit implementation that uses microinstructions stored in a control store to generate control signals.
Hazard	A condition that prevents the next instruction in the pipeline from executing during its designated clock cycle.
ILP (Instruction-Level Parallelism)	The degree to which instructions in a program can be executed in parallel.

Formulas (verbatim)

Formula	Statement
Speedup	Speedup = Old Time / New Time
Ideal pipeline speedup	Number of stages (5-stage → up to 5×).

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#3** 5-stage pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) (canonical)
Numericals	`Numerical-Book.md` **#1** CPI from mix
Tricky	`Top-10-Tricky-Concepts.md` **#6** multi-cycle vs single-cycle paradox
Top-100	`Top-100-Questions.md` **#26–#34** (pipeline basics)

Books · Chapters · Media

Resource	Where
Hennessy & Patterson CAQA	App C (pipeline intro)
Patterson & Hennessy COD	App C (microprogramming)
Sarangi (IITD)	Ch 9 (microprogramming) · Ch 10 (pipeline intro)

Fear-Killer Pack CA-W4 (verbatim)

Resources: P&H COD App C (microprogramming) • H&P CAQA App C (pipeline intro)

1. Differentiate horizontal vs vertical microprogramming. Design a microinstruction format for a control store with 64 microinstructions, 16 control signals, and a 6-way branch. Calculate the control store size.
2. Draw the 5-stage pipeline (IF, ID, EX, MEM, WB) with pipeline registers. Trace 3 instructions through it showing which stage each occupies per cycle.
3. What is the ideal speedup of a 5-stage pipeline? What prevents it in practice? Define structural, data, and control hazards in one sentence each.

Narrative — Week 4 (verbatim)

Topics: Microprogrammed control (horizontal vs vertical); Pipelining concept; 5-stage pipeline

Resources: Sarangi Ch. 9 (microprogramming), Ch. 10 (pipeline intro)

Practice: - Vertical vs horizontal microprogramming comparison table - Draw the 5-stage pipeline (IF, ID, EX, MEM, WB) - Trace 3 instructions through the pipeline showing each stage

Deliverable: 5-stage pipeline diagram + microprogramming comparison table

Time budget: 6 hrs

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q2 pipeline draw + Q1 control-store calc) · def/formula skim · same-problem drill #1
- **Same-problem drill target:** 5-stage pipeline draw < 15 min; horizontal vs vertical table from memory
- **Deliverable:** 5-stage pipeline diagram + microprogramming comparison table
- **Trap:** M4 gap — forwarding/hazard detection has no chapter breakdown; mapped at pack level

W5 — Pipeline Hazards I · 14–20 Sep · P0 · 7 hrs

Banner: Rotation CA (Mon+Tue) · Tier P0 · Time budget 7 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 5	lines 85–100
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W5	lines 42–49
Week manifest	`weeks/CA-W5.md`	full file
Chapter breakdown	`03-Chapter-Breakdowns/04-…`	embedded below

Definitions (verbatim)

Term	Definition
RAW Hazard	A true data dependence where an instruction reads a register that a previous instruction writes (Read After Write).
WAW (Write After Write)	A name dependence where two instructions write to the same register. Resolved by ensuring correct write order.
WAR (Write After Read)	A name dependence where an instruction writes a register that a previous instruction reads. Not a hazard in 5-stage MIPS pipeline (all reads happen in ID, writes in WB).

Forwarding (Bypassing)	A technique that resolves data hazards by feeding the ALU output from a later pipeline stage back to the ALU input of an earlier stage.
Hazard	A condition that prevents the next instruction in the pipeline from executing during its designated clock cycle.

Formulas (verbatim)

Formula	Statement
Effective CPI	Ideal CPI (1) + stall cycles per instruction from hazards.

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#4** pipeline with forwarding paths and muxes · **#5** hazard detection unit
Numericals	`Numerical-Book.md` **#13** hazard penalty
Tricky	`Top-10-Tricky-Concepts.md` **#2** EX vs MEM hazard priority · **#3** load-use stall
Top-100	`Top-100-Questions.md` **#28–#53** (hazards)

Books · Chapters · Media

Resource	Where
Patterson & Hennessy COD	Ch 4 (data/control hazards)
Hennessy & Patterson CAQA	App C
Sarangi (IITD)	Ch 10

Fear-Killer Pack CA-W5 (verbatim)

Resources: P&H COD Ch 4 (data/control hazards) • H&P CAQA App C

1. Given the sequence `lw $t0, 0($t1)` / `add $t2, $t0, $t3` / `sub $t4, $t2, $t5` / `sw $t4, 0($t6)`, identify every data hazard. Show which hazards are resolved by forwarding and which require a stall, with pipeline timing diagrams showing exactly when each instruction is in each stage.
2. Explain why WAR and WAW hazards do not occur in the standard 5-stage MIPS pipeline. Construct an out-of-order sequence where they would appear if the pipeline allowed it. Show the pipeline diagram.
3. A 5-stage pipeline has 30% branch instructions with 60% taken. Each misprediction costs 3 stalls. Structural hazards from the MEM unit occur on 10% of cycles costing 1 stall each. Calculate the effective CPI assuming ideal CPI = 1.
4. Why can't a structural hazard always be solved by adding more hardware? Give a concrete example where duplicating a resource creates more problems than it solves, including the cost and timing impact.
5. Trace the sequence `lw $t0, 0($t1)` / `add $t2, $t0, $t3` / `sub $t4, $t2, $t5` through the pipeline with forwarding paths drawn. Identify the exact cycle where each forwarding mux fires and which register values propagate.

Narrative — Week 5 (verbatim)

Topics: Structural hazards; Data hazards (RAW, WAW, WAR); Forwarding unit design

Resources: Sarangi Ch. 10; practice timing diagrams

Practice: - Identify RAW, WAW, WAR in 8 instruction sequences - Design the forwarding unit: show forwarding muxes, write forwarding conditions - **Memorize:** $zero register exception in forwarding logic

Killer trap: RAR is NOT a hazard. WAR happens from read-before-write (name says it). WAW = two writes in a row. If you confuse these, you lose 10 marks.

Deliverable: 5 hazard identification traces with forwarding paths drawn

Time budget: 7 hrs

Chapter 4 — Pipeline Hazards (verbatim)

Weight: ★★★★★

Difficulty: ★★★★★

Importance: ★★★★★

Learning Objectives

Explain

- Structural, data, and control hazards
- Forwarding vs stalling

Draw

- 5-stage pipeline diagram with forwarding paths
- Hazard detection unit
- Branch predictor state machine

Trace

- Multiple instructions through pipeline showing stalls/forwards
- 1-bit and 2-bit predictor accuracy

Design

- Forwarding unit logic (EX, MEM, WB priority)
- Hazard detection stall conditions

Must Memorize

RAW (Read After Write): A true data dependence where an instruction reads a register that a previous instruction writes. Forwarding can resolve most RAW hazards.

WAW (Write After Write): A name dependence where two instructions write to the same register. Resolved by ensuring correct write order.

WAR (Write After Read): A name dependence where an instruction writes a register that a previous instruction reads. Not a hazard in 5-stage MIPS pipeline (all reads happen in ID, writes in WB).

Forwarding (bypassing): Feeding the ALU output directly from EX/MEM or MEM/WB pipeline registers to the ALU inputs, avoiding stalls.

Must Draw

- 5-stage pipeline: IF, ID, EX, MEM, WB
- Forwarding muxes at ALU inputs
- Hazard detection unit

Common Mistakes

1. **RAR is NOT a hazard** — two reads, no conflict
2. **Forwarding $zero** — $zero is read but forwarding should not override it (trap)
3. **EX hazard vs MEM hazard priority** — EX forwarding has priority (more recent)
4. **Load-use hazard** — cannot forward from MEM to EX in same cycle (need 1 stall)
5. **Branch penalty with 2-bit predictor** — misprediction flips the state; always compute penalty correctly

Typical Questions

1. Identify RAW/WAW/WAR in instruction sequences
2. Draw forwarding paths for a 3-instruction sequence
3. Draw 2-bit predictor state machine and trace outcomes
4. Calculate speedup with pipelining given hazard frequency
5. Design hazard detection unit — when does it insert stalls?

GPA Priority: ▩ **Must Win**

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 hazard identification + Q5 trace) · def/formula skim · same-problem drill #13
- **Same-problem drill target:** identify RAW/WAW/WAR + forward/stall decision < 30 s per sequence
- **Deliverable:** 5 hazard identification traces with forwarding paths drawn
- **Trap:** RAR is NOT a hazard; WAR = read-before-write; WAW = two writes in a row

---

W6 — Pipeline Hazards II · 21–27 Sep · P0 · 7 hrs

Banner: Rotation CA (Mon+Tue) · Tier P0 · Time budget 7 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 6	lines 104–117
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W6	lines 51–58
Week manifest	`weeks/CA-W6.md`	full file
Chapter breakdown	`03-Chapter-Breakdowns/04-…`	reference (embedded W5)

Definitions (verbatim)

Term	Definition
Forwarding (Bypassing)	A technique that resolves data hazards by feeding the ALU output from a later pipeline stage back to the ALU input of an earlier stage.
Branch Prediction	A technique that predicts the outcome of a conditional branch before it is resolved, allowing the pipeline to continue without stalling.

Formulas (verbatim)

Formula	Statement
Branch penalty	P(mispredict) × penalty cycles per branch × branch frequency.

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#5** hazard detection unit · **#7** 1-bit predictor · **#8** 2-bit predictor
Numericals	`Numerical-Book.md` **#14** branch predictor accuracy · **#15** forwarding conditions
Tricky	`Top-10-Tricky-Concepts.md` **#1** $zero forwarding trap · **#2** hazard priority · **#4** 1-bit double misprediction
Top-100	`Top-100-Questions.md` **#36–#54** (predictors, OOO)

Books · Chapters · Media

Resource	Where
Patterson & Hennessy COD	Ch 4 (forwarding, branch prediction)
Hennessy & Patterson CAQA	App C
Sarangi (IITD)	Ch 10

Fear-Killer Pack CA-W6 (verbatim)

Resources: P&H COD Ch 4 (forwarding, branch prediction) • H&P CAQA App C

1. Design the complete forwarding unit with comparators and priority logic. Show EX/MEM forwarding vs MEM/WB forwarding, the exact comparison conditions, and explain why EX hazard takes priority over MEM hazard. Draw the circuit.
2. The `$zero` register must never be forwarded. Explain why this is a special case and implement the `$zero` check in the

forwarding unit logic. Show the Verilog/Boolean expression. What incorrect behavior occurs if you forget this check?

3. Trace a 2-bit saturating counter predictor on the branch outcomes: T, T, T, NT, T, NT, NT, NT. Show the state diagram transitions step by step and count mispredictions. Repeat with a 1-bit predictor and compare the accuracy.
4. A loop iterates 10 times. Explain why a 1-bit predictor mispredicts twice (first and last iteration) while a 2-bit predictor mispredicts only once. Now generalize: what happens if the loop iterates only 3 times?
5. Calculate the branch penalty for a 5-stage pipeline using a 2-bit predictor with 85% accuracy. Branch frequency is 25%, branch penalty on mispredict is 3 cycles, no delay slot.

Narrative — Week 6 (verbatim)

Topics: Hazard detection unit; Stall vs forward; Branch prediction (1-bit, 2-bit); Control hazards

Resources: Sarangi Ch. 10; past paper hazard sequences

Practice: - Design hazard detection unit: when does it stall instead of forwarding? - 1-bit predictor accuracy analysis (the "twice-misprediction" on loop exit) - 2-bit predictor state machine: draw and trace 10 branch outcomes

Deliverable: Hazard detection unit diagram + branch prediction accuracy for 5 scenarios

Time budget: 7 hrs

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 forwarding unit + Q3 predictor trace) · def/formula skim · same-problem drill #14, #15
- **Same-problem drill target:** 2-bit predictor trace of 10 outcomes without error; forwarding priority logic < 30 s
- **Deliverable:** hazard detection unit diagram + branch prediction accuracy for 5 scenarios
- **Trap:** $zero must never be forwarded; EX hazard beats MEM hazard

W7 — Midterm Revision · 28 Sep–04 Oct · P1 · 7 hrs

Banner: Rotation CA (Mon+Tue) · Tier P1 · Time budget 7 hrs · P0 floor: pack pass + def/formula skim + same-problem drill. **No new material.**

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 7	lines 121–137
Fear-Killer Pack	`Fear-Killer-Packs.md` → CA-W7	lines 60–66
Week manifest	`weeks/CA-W7.md`	full file
Chapter breakdowns	all of 01–04	reference (embedded W1, W2, W5)

Definitions (verbatim) — full W1–W6 sweep

Term	Definition
ISA	The interface between hardware and software, defining the instructions, registers, memory addressing, and data types that a processor supports.
Single-cycle Datapath	A processor implementation where every instruction executes in exactly one clock cycle, with the cycle time determined by the longest instruction (typically lw).
Multi-cycle Datapath	A processor implementation that breaks instruction execution into multiple steps.
Microprogrammed Control	A control unit implementation that uses microinstructions stored in a control store to generate control signals.
Hazard	A condition that prevents the next instruction in the pipeline from executing during its designated clock cycle.
RAW Hazard	A true data dependence where an instruction reads a register that a previous instruction writes (Read After Write).
Forwarding (Bypassing)	A technique that resolves data hazards by feeding the ALU output from a later pipeline stage back to the ALU input of an earlier stage.
Branch Prediction	A technique that predicts the outcome of a conditional branch before it is resolved, allowing the pipeline to continue without stalling.

ILP	The degree to which instructions in a program can be executed in parallel.

Formulas (verbatim)

Formula	Statement
CPU Time	CPU Time = Instruction Count × CPI × Cycle Time
Speedup	Speedup = Old Time / New Time

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#1–#8** full sweep (canonical)
Numericals	`Numerical-Book.md` **#1, #2, #13–#15** (review)
Tricky	`Top-10-Tricky-Concepts.md` all 10 (last pass before midterm)
Top-100	`Top-100-Questions.md` **#1–#55**

Books · Chapters · Media

Resource	Where
Weeks 1–6 packs	timed past papers

Fear-Killer Pack CA-W7 (verbatim)

Resources: Weeks 1-6 packs • timed past papers

1. Draw the multi-cycle datapath from BLANK paper in 12 minutes. Trace R-type, lw, sw, branch through it. Check accuracy against the key only after completing.
2. Design one complete FSM control unit from memory: state diagram for R-type, lw, sw, branch, with state encoding and every transition labeled.
3. Identify RAW/WAW/WAR in 5 instruction sequences. For each hazard, state the decision: forward, stall, or no action. Include at least one load-use and one $zero case.
4. Sit a full-length midterm past paper (2 hrs, closed book). Answer diagram/numerical questions first, theory second.

Narrative — Week 7 (verbatim)

Topics: Comprehensive review of Weeks 1-6

Practice: - Timed full-length past paper (2 hrs, closed book) - Multi-cycle datapath: draw from BLANK PAPER in 12 min. Check accuracy against key ONLY after completing. - FSM control: design one complete control unit from memory - Pipeline hazards: 3 sequences, identify + forward + stall decisions

Retrieval protocol: Do NOT re-read notes. Each practice task must start from a blank page. Check answers only after completing. This forces the testing effect.

Red flag: If you cannot draw the multi-cycle datapath in <15 min, repeat 5 times daily until exam day. Repeat the same process for midterm exam week mornings.

Sleep banking: Bedtime moves to 22:00 for 5 nights (Weeks 6-7). Sleep 9 hours. This protects memory consolidation of datapath design, FSM control, and pipeline hazards during hippocampal replay in deep sleep.

Time budget: 7 hrs

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 timed draw + Q2 FSM from memory) · def/formula skim · same-problem drill #1/#2
- **Same-problem drill target:** multi-cycle datapath < 12 min; FSM complete with every transition to state 0
- **Deliverable:** full-length midterm past paper solved and self-graded
- **Trap:** retrieval only — no passive re-reading; if datapath draw > 15 min, drill it 5× daily

---

W8 — MIDTERM EXAM WEEK · 05–11 Oct · Exam

Banner: Rotation CA (Mon+Tue) · Tier MIDTERM · No new deep study, no floor accrual. Ledger frozen during W8.

Sources

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 8	lines 141–149
Pack	`Fear-Killer-Packs.md` → Week 8	lines 68–69
Week manifest	`weeks/CA-W8.md`	full file

Fear-Killer Pack — Week 8 (verbatim)

Week 8: MIDTERM EXAM WEEK — No new pack. Active recall only — review the 1-page cheat sheet and sleep 8 hours.

Narrative — Week 8 (verbatim)

Focus: No new material. Active recall only.

- Review 1-page cheat sheet (prepared Week 7)
- Sleep 8 hours each night
- During exam: Answer diagram/numerical questions first, theory second
- Check every control signal, every hazard classification

Exam-day stack (Mon–Fri)

- [ ] Past-paper run for the exam subject — 60 min, P0
- [ ] Blank-page retrieval of that subject — 30 min, P0
- [ ] Master Error Log review — 20 min, P0

Notes

- Review the 1-page cheat sheet (prepared Wk7). Sleep 8 h each night.
- Exam order: diagram/numerical questions first, theory second.
- **Ledger frozen during Wk8** — no new accrual; cleared in Wk9 recovery.